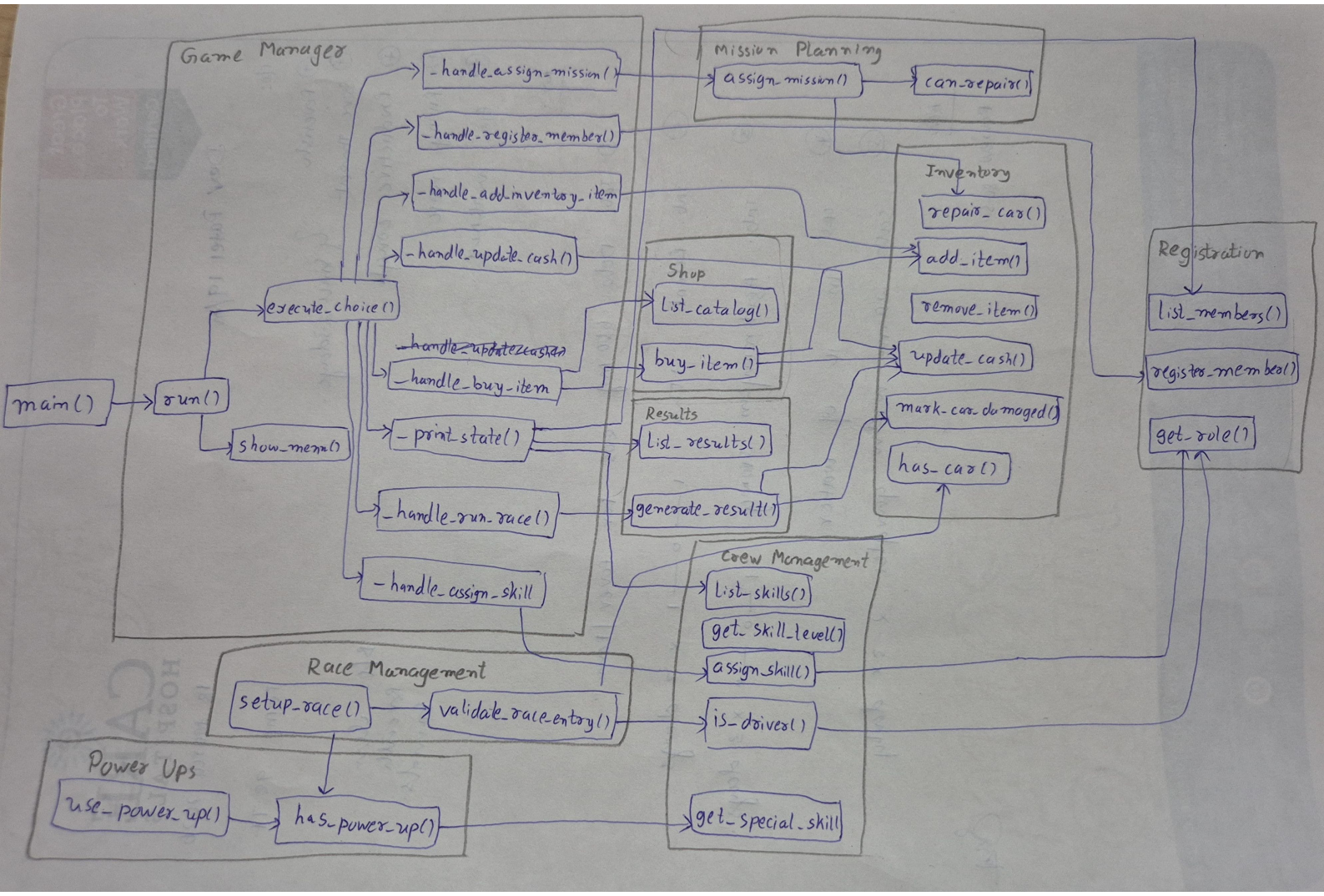
Game Manager
main()
run()
show_menu()
execute_choice()
_handle_assign_mission()
_handle_register_member()
_handle_add_inventory_item
_handle_update_cash()
~~handle_updatecash~~
_handle_buy_item
_print_state()
_handle_run_race()
_handle_assign_skill
Mission Planning
assign_mission()
can_repair()
Shop
List_catalog()
buy_item()
Results
List_results()
generate_result()
Inventory
repair_car()
add_item()
remove_item()
update_cash()
mark_car_damaged()
has_car()
Registration
list_members()
register_member()
get_role()
Crew Management
List_skills()
get_skill_level()
assign_skill()
is_driver()
get_special_skill
Race Management
setup_race()
validate_race_entry()
Power Ups
use_power_up()
has_power_up()

Extra modules in Street Race Manager

1. Shop Module

- Purpose: Handles catalog lookup and purchase transactions for race-related items.
- Integration role: It is the entry point for buying new cars, which are required by Race Management for valid race setup.
- Main interactions: Updates InventoryModule cash and owned items when purchases succeed. also blocks transactions when cash is insufficient or item is invalid.

2. Power Ups Module

- Purpose: Validates whether a driver can use a named power-up in race setup.
- Integration role: It connects Crew Management skill assignment to Race Management setup validation.
- Main interactions: Reads driver power-up capability from CrewManagementModule data and flags whether the power-up is applied in race context.

Integration Testing Report

- Test type: Integration testing
- Test file: tests/integration_test.py
- Total tests 22. 13 failed, 9 passed (before any fixes)
- Current system (after bug fixes) result: 22 passed

Test Cases

1) test_registration_failure_blocks_skill_assignment

- Why this test is needed: It confirms invalid registration cannot be used later in crew setup.
- Scenario: Register with invalid role, then assign skill to same member.
- Modules involved: Registration, Crew Management.
- Expected result: Registration fails, then skill assignment also fails.
- Actual result after testing (old system): Passed.

2) test_invalid_skill_input_prevents_race_setup

- Why this test is needed: It validates edge input handling for skill and checks downstream race integration behavior.
- Scenario: Skill is non-integer; car exists; race setup is called.
- Modules involved: Registration, Crew Management, Inventory, Race Management.
- Expected result: Skill assignment fails; race setup still succeeds when driver role and car checks pass.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Unified race setup to accept integration payload format used by flow and tests.

3) test_unknown_shop_item_does_not_change_inventory_or_cash

- Why this test is needed: It ensures invalid shopping input does not corrupt money/inventory state.
- Scenario: Buy unknown item from shop.
- Modules involved: Shop, Inventory.
- Expected result: Purchase rejected; cash and cars unchanged.
- Actual result after testing (old system): Passed.

4) test_low_cash_prevents_buy_and_then_race_fails_for_missing_car

- Why this test is needed: It checks two connected failures: low-cash purchase block and race dependency on owned car.
- Scenario: Cash too low for car purchase, then race with missing car.
- Modules involved: Shop, Inventory, Registration, Crew Management, Race Management.
- Expected result: Buy fails; race setup fails because car unavailable.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch before business validation.
- Fix applied: Race setup interface aligned with integration payload format.

5) test_race_rejects_empty_driver_and_car_names

- Why this test is needed: It validates empty-input edge cases at race entry point.
- Scenario: Blank driver and blank car in race request.
- Modules involved: Race Management.
- Expected result: Setup rejected with required-input message.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Updated race setup call contract to accept request/reference payload and reach validation logic.

6) test_race_rejects_non_driver_even_with_car

- Why this test is needed: It enforces role-based rule across modules.
- Scenario: Mechanic with a car tries to start race.
- Modules involved: Registration, Crew Management, Inventory, Race Management.
- Expected result: Setup rejected because member is not a driver.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Standardized race setup integration interface.

7) test_race_rejects_driver_with_unowned_car

- Why this test is needed: It verifies inventory dependency in race setup.
- Scenario: Driver exists but car is not in inventory.
- Modules involved: Registration, Crew Management, Inventory, Race Management.
- Expected result: Setup rejected for missing car.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Race setup now accepts integration payload, enabling correct car-availability validation.

8) test_blank_powerup_is_not_applied_in_successful_race_setup

- Why this test is needed: It checks blank-input edge case for power-up path.
- Scenario: Valid driver+car, but whitespace power-up.
- Modules involved: Registration, Crew Management, Inventory, Power-Ups, Race Management.
- Expected result: Setup succeeds and power-up is not applied.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Race setup contract aligned so power-up logic executes correctly.

9) test_powerup_match_is_case_insensitive_in_race_setup

- Why this test is needed: It validates realistic text-input variation in integration path.
- Scenario: Stored power-up and input differ by case.
- Modules involved: Registration, Crew Management, Inventory, Power-Ups, Race Management.
- Expected result: Power-up should still be applied.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Consistent request/reference race setup integration path.

10) test_forced_win_updates_cash_and_history

- Why this test is needed: It validates outcome-to-state propagation across race and finance history.
- Scenario: Deterministic winning roll after setup.
- Modules involved: Registration, Crew Management, Shop, Inventory, Race Management, Results.
- Expected result: Cash increases by prize and history count increments.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Fixed race setup interface used in integration flow.

11) test_forced_loss_marks_car_damaged

- Why this test is needed: It verifies loss path side effects that later repair missions depend on.
- Scenario: Deterministic losing roll.
- Modules involved: Inventory, Results.
- Expected result: Car marked as damaged.
- Actual result after testing (old system): Passed.

12) test_repair_fails_without_mechanic_after_loss

- Why this test is needed: It ensures mission role checks work after a real race outcome.
- Scenario: Car damaged after loss, no mechanic registered.
- Modules involved: Registration, Crew Management, Inventory, Results, Mission Planning.
- Expected result: Repair mission rejected.
- Actual result after testing (old system): Passed.

13) test_repair_succeeds_with_mechanic_after_loss

- Why this test is needed: It confirms complete recovery path works after damage.
- Scenario: Car damaged after loss, mechanic present.
- Modules involved: Registration, Crew Management, Inventory, Results, Mission Planning.
- Expected result: Repair mission succeeds and damage cleared.
- Actual result after testing (old system): Passed.

14) test_repair_fails_when_target_not_damaged_even_with_mechanic

- Why this test is needed: It protects against invalid repair requests.
- Scenario: Mechanic exists but target car is not damaged.
- Modules involved: Registration, Inventory, Mission Planning.
- Expected result: Repair mission rejected.
- Actual result after testing (old system): Passed.

15) test_non_repair_mission_succeeds_with_minimal_requirements

- Why this test is needed: It checks non-repair mission flow does not over-validate unrelated constraints.
- Scenario: Assign a non-repair mission.
- Modules involved: Mission Planning, Registration, Inventory.
- Expected result: Mission accepted.
- Actual result after testing (old system): Passed.

16) test_inventory_invalid_category_blocks_tooling_for_flow

- Why this test is needed: It validates category edge-case input used in setup flows.
- Scenario: Add inventory item with invalid category.
- Modules involved: Inventory.
- Expected result: Request rejected.
- Actual result after testing (old system): Passed.

17) test_end_to_end_buy_lose_repair_flow

- Why this test is needed: It validates a realistic long workflow covering many module handoffs.
- Scenario: Register crew, buy car, lose race, repair car.
- Modules involved: Registration, Crew Management, Shop, Inventory, Race Management, Results, Mission Planning.
- Expected result: Complete flow succeeds and final car state is repaired.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Race setup integration contract fixed; workflow now runs end to end.

18) test_end_to_end_buy_and_win_flow

- Why this test is needed: It validates positive full workflow and final money state.
- Scenario: Register driver, buy car, win race.
- Modules involved: Registration, Crew Management, Shop, Inventory, Race Management, Results.
- Expected result: Final cash reflects purchase and reward correctly.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Race setup integration contract fixed; flow now completes and cash updates correctly.

19) test_register_driver_then_enter_race_success

- Why this test is needed: It directly verifies the core call-graph integration from registration to race setup.
- Scenario: Register a valid driver, assign skill, add a car, and enter race.
- Modules involved: Registration, Crew Management, Inventory, Race Management.
- Expected result: Race setup succeeds and context is returned.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Race setup accepts integration payload structure and reaches normal validation/success path.

20) test_enter_race_without_registered_driver_fails

- Why this test is needed: It ensures race entry is blocked when registration prerequisite is missing.
- Scenario: Try to start race for a name that was never registered.
- Modules involved: Registration, Inventory, Race Management.
- Expected result: Race setup is rejected with registration/driver validation message.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch.
- Fix applied: Race setup contract updated so integration validation can execute and reject correctly.

21) test_complete_race_updates_results_and_prize_money_inventory

- Why this test is needed: It verifies post-race integration where results history and inventory cash both update together.
- Scenario: Register and prepare driver, run race with forced win, validate result log and cash change.
- Modules involved: Registration, Crew Management, Shop, Inventory, Race Management, Results.
- Expected result: One result is stored and prize money updates inventory cash balance.
- Actual result after testing (old system): Failed.
- Error/Bug found in old system: TypeError from old `setup_race` interface mismatch prevented race completion path.
- Fix applied: Race setup integration interface fixed, enabling result generation and prize-money update path.

22) test_assign_mission_validates_crew_roles_for_repair

- Why this test is needed: It confirms mission assignment enforces correct crew role requirements from call-graph interactions.
- Scenario: Attempt repair mission without mechanic, then register mechanic and retry.
- Modules involved: Registration, Inventory, Mission Planning.
- Expected result: First repair assignment fails (no mechanic), second succeeds after mechanic registration.
- Actual result after testing (old system): Passed.